What is Exploratory Data Analysis

Exploratory Data Analysis is a data analytics process to understand the data in depth and learn the different data characteristics, often with visual means. This allows you to get a better feel of your data and find useful patterns in it.

Figure 1: Exploratory Data Analysis

It is crucial to understand it in depth before you perform data analysis and run your data through an algorithm. You need to know the patterns in your data and determine which variables are important and which do not play a significant role in the output. Further, some variables may have correlations with other variables. You also need to recognize errors in your data.

All of this can be done with Exploratory Data Analysis. It helps you gather insights and make better sense of the data, and removes irregularities and unnecessary values from data.

- Helps you prepare your dataset for analysis.
- Allows a machine learning model to predict our dataset better.
- Gives you more accurate results.

- It also helps us to choose a better machine learning model.

Figure 2: Exploratory Data Analysis uses

Steps invloved in Exploratory Data Analysis

Data Collection

Data collection is an essential part of exploratory data analysis. It refers to the process of finding and loading data into our system. Good, reliable data can be found on various public sites or bought from private organizations. Some reliable sites for data collection are Kaggle, Github, Machine Learning Repository, etc.

Data Cleaning

refers to the process of removing unwanted variables and values from your dataset and getting rid of any irregularities in it. Such anomalies can disproportionately skew the data and hence adversely affect the results. Some steps that can be done to clean data are :

- Removing missing values, outliers, and unnecessary rows/ columns.

- Re-indexing and reformatting our data.

Univariate Analysis

In Univariate Analysis, you analyze data of just one variable. A variable in your dataset refers to a single feature/ column. You can do this either with graphical or non-graphical means by finding specific mathematical values in the data. Some visual methods include:

- Histograms: Bar plots in which the frequency of data is represented with rectangle bars.

- Box-plots: Here the information is represented in the form of boxes.

Bivariate Analysis

Here, you use two variables and compare them. This way, you can find how one feature affects the other. It is done with scatter plots, which plot individual data points or correlation matrices that plot the correlation in hues. You can also use boxplots.

for eg plot a **scatter plot** of the *greater living area* and *Sales prices*

NOW WE PERFORM OUR FIRST ANALYSIS WITH EDA

Market Analysis with EDA

In [2]:
```
import pandas as pd
import numpy as np
from matplotlib import pyplot as plt
import seaborn as sns
data=pd.read_csv('market_analysis.csv',skiprows=2)# before it was seendataset is not formatted co
#The first two rows contain the actual column names, and the column names are just arbitrary valu

#To overcome the skewed rows, import your data by skipping the first two rows.
#This will make sure that your column names are populated correctly.

# do this by using skiprows
data.head(15)
```

Out[2]:

	customerid	age	salary	balance	marital	jobedu	targeted	default	housing
0	1	58.0	100000	2143	married	management,tertiary	yes	no	yes
1	2	44.0	60000	29	single	technician,secondary	yes	no	yes
2	3	33.0	120000	2	married	entrepreneur,secondary	yes	no	yes
3	4	47.0	20000	1506	married	blue-collar,unknown	no	no	yes
4	5	33.0	0	1	single	unknown,unknown	no	no	no
5	6	35.0	100000	231	married	management,tertiary	yes	no	yes
6	7	28.0	100000	447	single	management,tertiary	no	no	yes
7	8	42.0	120000	2	divorced	entrepreneur,tertiary	no	yes	yes
8	9	58.0	55000	121	married	retired,primary	yes	no	yes
9	10	43.0	60000	593	single	technician,secondary	yes	no	yes
10	11	41.0	50000	270	divorced	admin.,secondary	yes	no	yes
11	12	29.0	50000	390	single	admin.,secondary	yes	no	yes
12	13	53.0	60000	6	married	technician,secondary	yes	no	yes
13	14	58.0	60000	71	married	technician,unknown	no	no	yes
14	15	57.0	70000	162	married	services,secondary	yes	no	yes

now that we have correctly imported our dataset,lets move to data cleaning

Data cleaning

In [3]:
```
# let us drop the customer id column as it is nothing just rows number
```

In [4]:
```
data.drop('customerid',axis=1,inplace=True)
data.head(15)
```

Out[4]:

	age	salary	balance	marital	jobedu	targeted	default	housing	loan	con
0	58.0	100000	2143	married	management,tertiary	yes	no	yes	no	unkn
1	44.0	60000	29	single	technician,secondary	yes	no	yes	no	unkn
2	33.0	120000	2	married	entrepreneur,secondary	yes	no	yes	yes	unkn
3	47.0	20000	1506	married	blue-collar,unknown	no	no	yes	no	unkn
4	33.0	0	1	single	unknown,unknown	no	no	no	no	unkn
5	35.0	100000	231	married	management,tertiary	yes	no	yes	no	unkn
6	28.0	100000	447	single	management,tertiary	no	no	yes	yes	unkn
7	42.0	120000	2	divorced	entrepreneur,tertiary	no	yes	yes	no	unkn
8	58.0	55000	121	married	retired,primary	yes	no	yes	no	unkn
9	43.0	60000	593	single	technician,secondary	yes	no	yes	no	unkn
10	41.0	50000	270	divorced	admin.,secondary	yes	no	yes	no	unkn
11	29.0	50000	390	single	admin.,secondary	yes	no	yes	no	unkn
12	53.0	60000	6	married	technician,secondary	yes	no	yes	no	unkn
13	58.0	60000	71	married	technician,unknown	no	no	yes	no	unkn
14	57.0	70000	162	married	services,secondary	yes	no	yes	no	unkn

In [5]:
```
# Also, split the 'jobedu' column into two. One column for the job and one for the education field
```

In [7]:
```
data['job']=data["jobedu"].apply(lambda x: x.split(",")[0])
data['education']=data["jobedu"].apply(lambda x: x.split(",")[1])
data.head(15)
```

Out[7]:

	age	salary	balance	marital	jobedu	targeted	default	housing	loan	con
0	58.0	100000	2143	married	management,tertiary	yes	no	yes	no	unkn
1	44.0	60000	29	single	technician,secondary	yes	no	yes	no	unkn
2	33.0	120000	2	married	entrepreneur,secondary	yes	no	yes	yes	unkn
3	47.0	20000	1506	married	blue-collar,unknown	no	no	yes	no	unkn
4	33.0	0	1	single	unknown,unknown	no	no	no	no	unkn
5	35.0	100000	231	married	management,tertiary	yes	no	yes	no	unkn
6	28.0	100000	447	single	management,tertiary	no	no	yes	yes	unkn
7	42.0	120000	2	divorced	entrepreneur,tertiary	no	yes	yes	no	unkn
8	58.0	55000	121	married	retired,primary	yes	no	yes	no	unkn
9	43.0	60000	593	single	technician,secondary	yes	no	yes	no	unkn
10	41.0	50000	270	divorced	admin.,secondary	yes	no	yes	no	unkn
11	29.0	50000	390	single	admin.,secondary	yes	no	yes	no	unkn
12	53.0	60000	6	married	technician,secondary	yes	no	yes	no	unkn
13	58.0	60000	71	married	technician,unknown	no	no	yes	no	unkn
14	57.0	70000	162	married	services,secondary	yes	no	yes	no	unkn

In [8]:
```
# After splitting the columns, you can drop the 'jobedu' column as it is of no use anymore.
```

In [9]:
```
data.drop('jobedu',axis=1,inplace=True)
data.head(15)
```

Out[9]:

	age	salary	balance	marital	targeted	default	housing	loan	contact	day	month	dur
0	58.0	100000	2143	married	yes	no	yes	no	unknown	5	may, 2017	26
1	44.0	60000	29	single	yes	no	yes	no	unknown	5	may, 2017	15
2	33.0	120000	2	married	yes	no	yes	yes	unknown	5	may, 2017	7
3	47.0	20000	1506	married	no	no	yes	no	unknown	5	may, 2017	9
4	33.0	0	1	single	no	no	no	no	unknown	5	may, 2017	19
5	35.0	100000	231	married	yes	no	yes	no	unknown	5	may, 2017	13
6	28.0	100000	447	single	no	no	yes	yes	unknown	5	may, 2017	21
7	42.0	120000	2	divorced	no	yes	yes	no	unknown	5	may, 2017	38
8	58.0	55000	121	married	yes	no	yes	no	unknown	5	may, 2017	5
9	43.0	60000	593	single	yes	no	yes	no	unknown	5	may, 2017	5
10	41.0	50000	270	divorced	yes	no	yes	no	unknown	5	may, 2017	22
11	29.0	50000	390	single	yes	no	yes	no	unknown	5	may, 2017	13
12	53.0	60000	6	married	yes	no	yes	no	unknown	5	may, 2017	51
13	58.0	60000	71	married	no	no	yes	no	unknown	5	may, 2017	7
14	57.0	70000	162	married	yes	no	yes	no	unknown	5	may, 2017	17

Missing values

In [10]:
```
# check missing values
```

In [11]:
```
data.isnull().sum()
```

Out[11]:
```
age          20
salary        0
balance       0
marital       0
targeted      0
default       0
housing       0
loan          0
contact       0
day           0
month        50
duration      0
campaign      0
pdays         0
previous      0
poutcome      0
response     30
job           0
education     0
dtype: int64
```

In [12]:
```
# There is nothing you can do about the missing age values. So, drop all rows which do not have the
```

In [13]:
```
data=data[~data.age.isnull()].copy()# drop all records not having age
data.isnull().sum()
```

Out[13]:
```
age          0
salary       0
balance      0
marital      0
targeted     0
default      0
housing      0
loan         0
contact      0
day          0
month       50
duration     0
campaign     0
pdays        0
previous     0
poutcome     0
response    30
job          0
education    0
dtype: int64
```

In [14]:
```
#Now, coming to the month column, you can fill in the missing values by finding the most common
#You see the mode of the month column to get the most commonly occurring values and fill in the
#using the fillna function.
```

In [16]:
```
month_mode=data.month.mode()[0]
print(month_mode)
```

```
may, 2017
```

In [17]:
```
data.month.fillna(month_mode,inplace=True)# fill missing month values with most occring/mode
# lets see month null values now
data.isnull().sum()
```

Out[17]:
```
age           0
salary        0
balance       0
marital       0
targeted      0
default       0
housing       0
loan          0
contact       0
day           0
month         0
duration      0
campaign      0
pdays         0
previous      0
poutcome      0
response     30
job           0
education     0
dtype: int64
```

In [18]:
```
#Finally, only the response column has missing values. You cannot do anything about these values.
#If the user hasn't filled in the response, you cannot auto-generate them. So you drop these values.
```

In [19]:
```
data=data[~data.response.isnull()].copy()# drop all records not having response
data.isnull().sum()
```

Out[19]:
```
age          0
salary       0
balance      0
marital      0
targeted     0
default      0
housing      0
loan         0
contact      0
day          0
month        0
duration     0
campaign     0
pdays        0
previous     0
poutcome     0
response     0
job          0
education    0
dtype: int64
```

now our data is cleaned

Handling Outliers

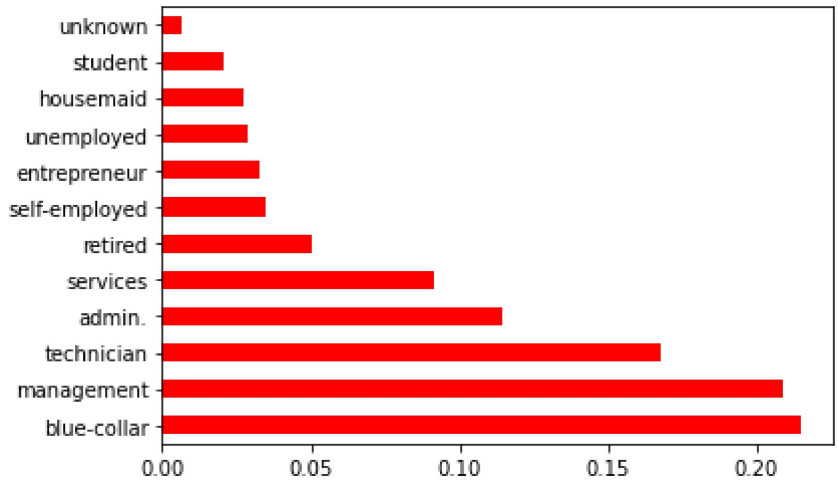
There are two types of outliers:

1. Univariate outliers: Univariate outliers are the data points whose values lie outside the expected range of values. Here, only a single variable is being considered.
2. Multivariate outliers: These outliers are dependent on the correlation between two variables. While plotting data, one variable may not lie beyond the expected range, but when you plot the same variable with some other variable, these values may lie far from the expected value.

Univariate Analysis

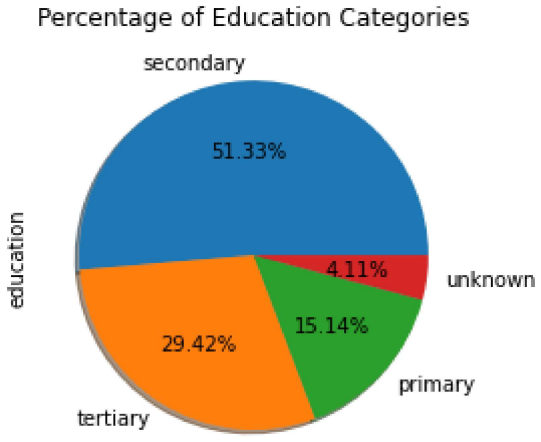
Now, consider the different jobs that you have data on. Plotting the job column as a bar graph in ascending order of the number of people who work in that job tells us the most popular jobs in the market. To ensure that they lie in the same range and are comparable, normalize the data.

In [22]:
```
data.job.value_counts(normalize=True)# calculating percentage of each job category
# plotting bar graph  of percentage job categories
data.job.value_counts(normalize=True).plot.barh(color='red')
plt.show()
```



Moving on, plot a pie chart to compare the education qualifications of the people in the survey. Almost half of the people have only a secondary school education and one-fourth have a tertiary education.

In [25]:
```
data.education.value_counts(normalize=True)# calculating percentage of each education category
# plotting pie chart  of percentage education categories
data.education.value_counts(normalize=True).plot.pie(autopct='%.2f%%',shadow=True,#startang
    )
plt.title('Percentage of Education Categories')
plt.show()
```



Bivariate analysis

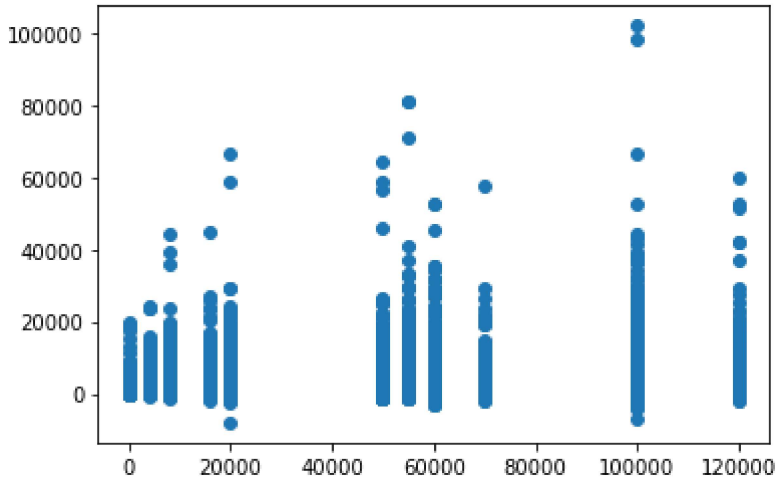
Is of 3 main types ;

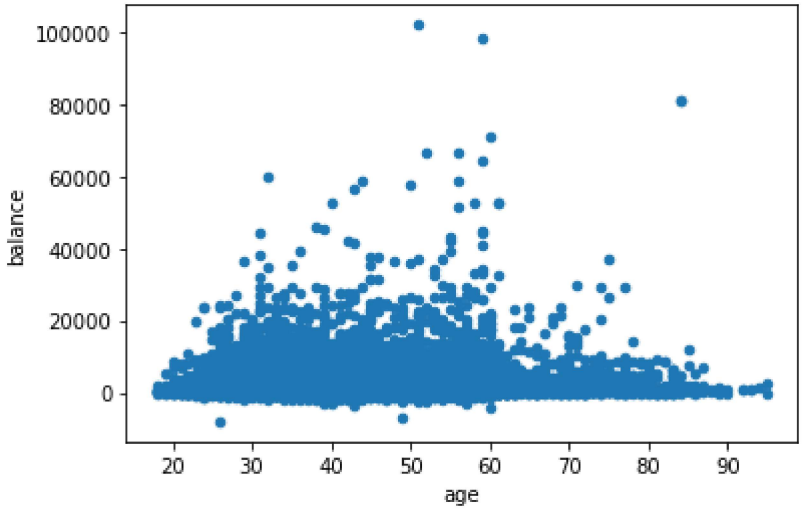
Numeric-Numeric Analysis

When both the variables being compared have numeric data, the analysis is said to be Numeric-Numeric Analysis. To compare two numeric columns, you can use scatter plots, pair plots, and correlation matrices.

In [ ]:
```
#scatter plot
 #A scatter plot is used to represent every data point in the graph.
#It shows how the data of one column fluctuates according to the corresponding data points in ano
#Plot a scatterplot between different individuals' salaries and bank balances and the balance and a
```

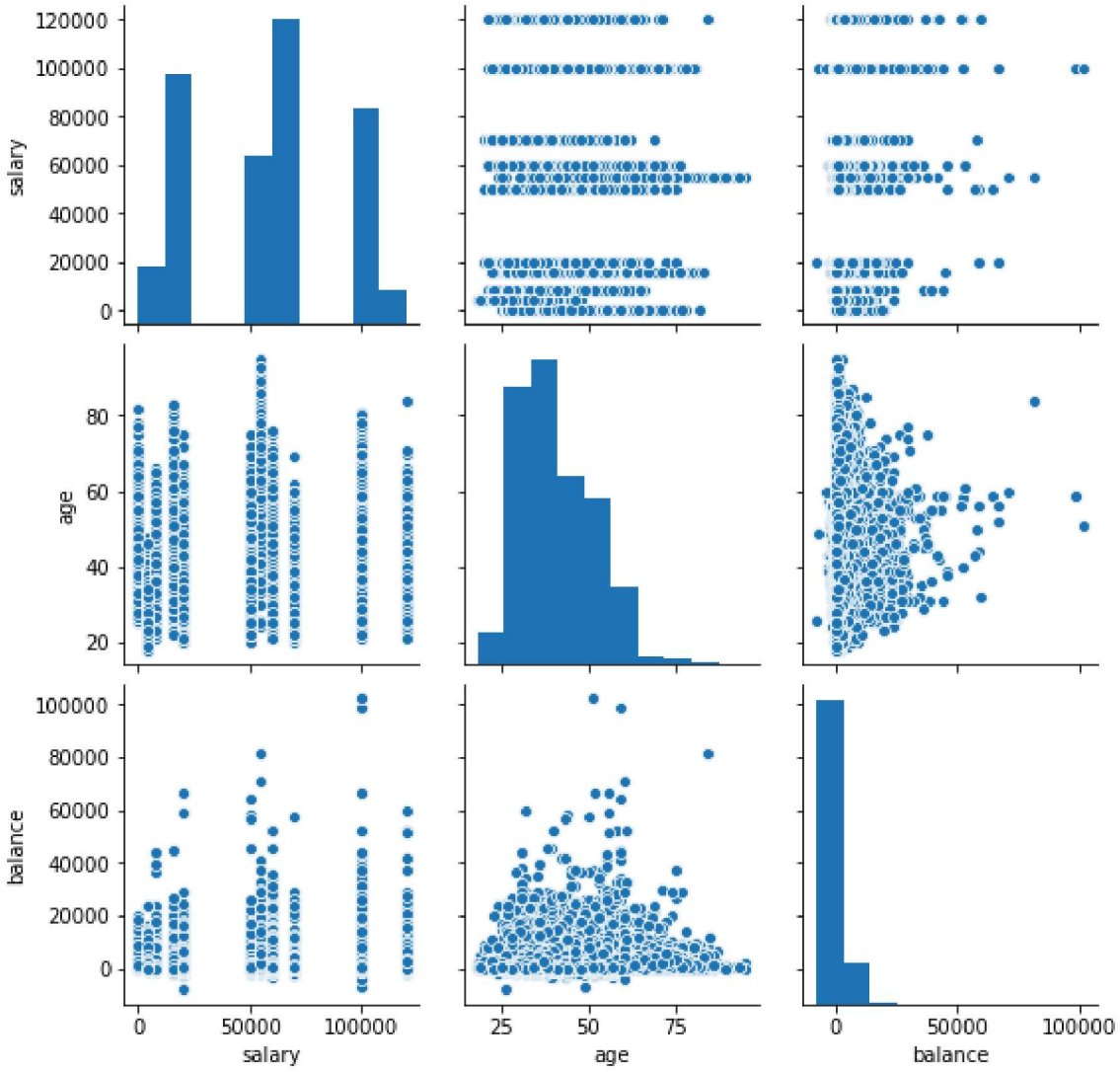
In [28]:
```
plt.scatter(data.salary,data.balance)# scatter plot of balance and salary variable
plt.show()
data.plot.scatter(x="age",y="balance")# scatter plot of balance and salary variable
plt.show()
```





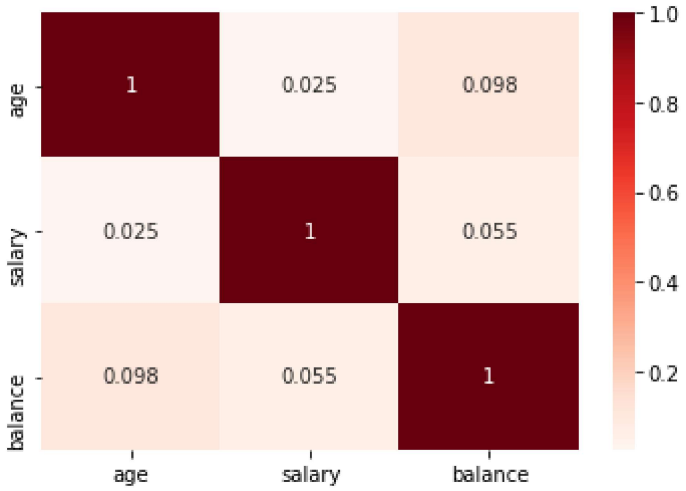
In [29]:
```
# pairplot
#Pair plots are used to compare multiple variables at the same time.
#They plot a scatter plot of all input variables against each other.
#This helps save space and lets us compare various variables at the same time.
#Let's plot the pair plot for salary, balance, and age.
```

In [30]:
```
sns.pairplot(data=data,vars=['salary','age','balance'])
plt.show()
```



In [31]:
```
# correlation matrix
#used to see the correlation between different variables.
#How correlated two variables are is determined by the correlation coefficient.
#The below table shows the correlation between salary, age, and balance.
#Correlation tells you how one variable affects the other.
#This helps us determine how changes in one variable will also cause a change in the other variable
```

In [32]:
```
data[['age','salary','balance']].corr()# creates matrix using age,salary,balance as rows and columns
sns.heatmap(data[['age','salary','balance']].corr(),annot=True,cmap='Reds')
plt.show()
```



The above matrix tells us that balance, age, and salary have a high correlation coefficient and affect each other. Age and salary have a lower correlation coefficient.

Numeric-Categorical Analysis

When one variable is of numeric type and another is a categorical variable, then you perform numeric-categorical analysis.

You can use the groupby function to arrange the data into similar groups. Rows that have the same value in a particular column will be arranged in a group together. This way, you can see the numerical occurrences of a certain category across a column. You can groupby values and find their mean.

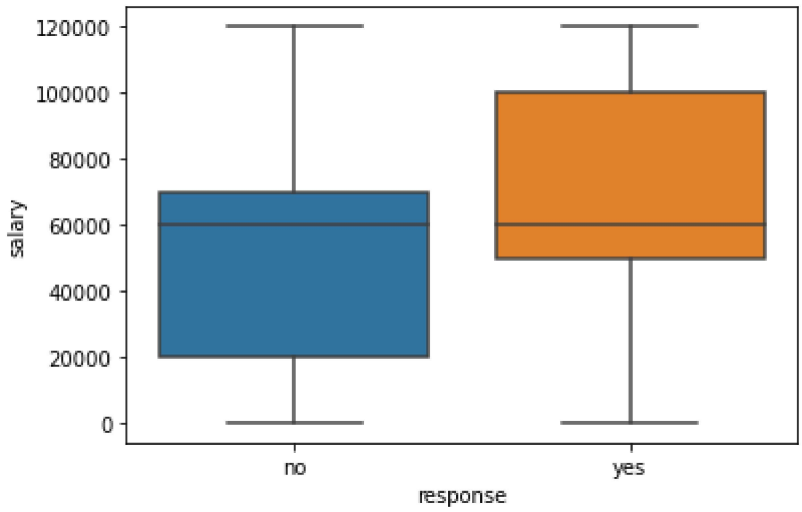
In [33]:
```
# groupby the response to find mean of salary with response yes,no
data.groupby('response')['salary'].mean()
#The values tell you the average salary of the people who have responded with yes and no in the re
```

Out[33]:
```
response
no     56769.510482
yes    58780.510880
Name: salary, dtype: float64
```

In [34]:
```
# groupby the response to find median or middle value of salary with response yes,no
data.groupby('response')['salary'].median()
```

Out[34]:
```
response
no     60000
yes    60000
Name: salary, dtype: int64
```

In [35]:
```
# plot box plot of salary for yes,no responses
sns.boxplot(data.response,data.salary)
plt.show()
```



The above plot tells you that the salary range of people who said no on the survey is between 20k - 70k with a median salary of 60k, while the salary range of people who replied with yes on the survey was between 50k - 100k with a median salary of 60K.

Categorical-Categorical Analysis

When both the variables contain categorical data, you perform categorical-categorical analysis. First, convert the categorical response column into a numerical column with 1 corresponding to a positive response and 0 corresponding to a negative response.

In [36]:
```
data['response_rate']=np.where(data.response=='yes',1,0)# yes=1,no=1
data.response_rate.value_counts()
```

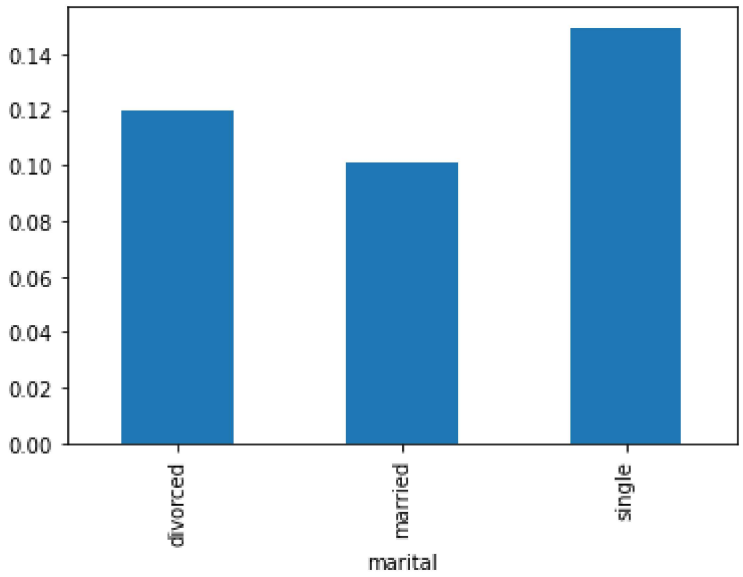
Out[36]:
```
0    39876
1     5285
Name: response_rate, dtype: int64
```

In [42]:
```
data.groupby('marital')['response_rate'].mean()

# tells  avg responses rate  how many are divorced,married,single
```

Out[42]:
```
marital
divorced    0.119469
married     0.101269
single      0.149554
Name: response_rate, dtype: float64
```

In [43]:
```
# now plot marital status with avg/mean number of response rate
data.groupby('marital')['response_rate'].mean().plot.bar()
plt.show()
```

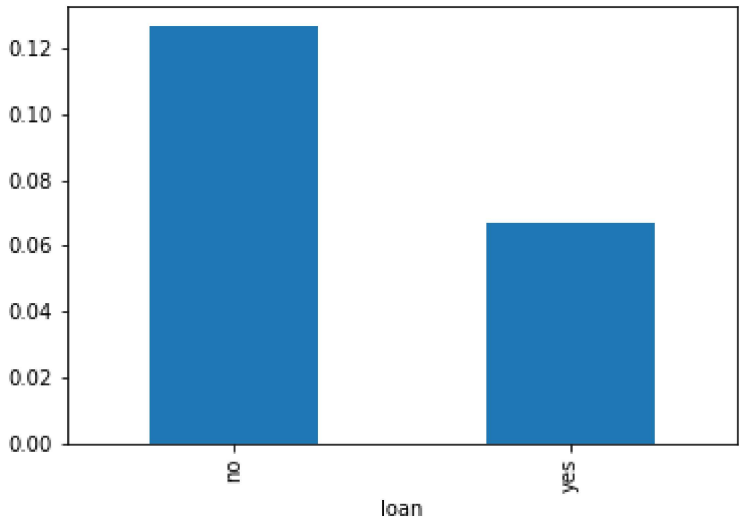


In [44]:
```
data.groupby('loan')['response_rate'].mean()
# # tells  avg responses yes,no how many took loan
```

Out[44]:
```
loan
no     0.126569
yes    0.066953
Name: response_rate, dtype: float64
```

In [45]:
```
data.groupby('loan')['response_rate'].mean().plot.bar()

plt.show()
```



SO THIS WAS OUR FIRST EXPLORATORY DATA ANALYSIS ....more will be done

EDA ON EMPLOYEES DATASET

In [52]:
```
import pandas as pd
import numpy as np
import seaborn as sns
from matplotlib import pyplot as plt
# read datasdet using pandas
data2 = pd.read_csv('employees.csv')
data2.head()
```

Out[52]:

	First Name	Gender	Start Date	Last Login Time	Salary	Bonus %	Senior Management	Team
0	Douglas	Male	8/6/1993	12:42 PM	97308	6.945	True	Marketing
1	Thomas	Male	3/31/1996	6:53 AM	61933	4.170	True	NaN
2	Maria	Female	4/23/1993	11:17 AM	130590	11.858	False	Finance
3	Jerry	Male	3/4/2005	1:00 PM	138705	9.340	True	Finance
4	Larry	Male	1/24/1998	4:47 PM	101004	1.389	True	Client Services

data2.info()

In [53]:
```
data2.describe()
```

Out[53]:

	Salary	Bonus %
count	1000.000000	1000.000000
mean	90662.181000	10.207555
std	32923.693342	5.528481
min	35013.000000	1.015000
25%	62613.000000	5.401750
50%	90428.000000	9.838500
75%	118740.250000	14.838000
max	149908.000000	19.944000

Changing Dtype from Object to Datetime

In [55]:
```
#Start Date is an important column for employees. However,
#it is not of much use if we can not handle it properly to handle this type of data pandas provide a s
#function datetime() from which we can change object type to DateTime format.
# convert "Start Date" column to datetime data type
data2['Start Date'] = pd.to_datetime(data2['Start Date'])
data2.head()
```

Out[55]:

	First Name	Gender	Start Date	Last Login Time	Salary	Bonus %	Senior Management	Team
0	Douglas	Male	1993-08-06	12:42 PM	97308	6.945	True	Marketing
1	Thomas	Male	1996-03-31	6:53 AM	61933	4.170	True	NaN
2	Maria	Female	1993-04-23	11:17 AM	130590	11.858	False	Finance
3	Jerry	Male	2005-03-04	1:00 PM	138705	9.340	True	Finance
4	Larry	Male	1998-01-24	4:47 PM	101004	1.389	True	Client Services

We can see the number of unique elements in our dataset. This will help us in deciding which type of encoding to choose for converting categorical columns into numerical columns.

In [56]:
```
data2.nunique()
```

Out[56]:
```
First Name           200
Gender                 2
Start Date           972
Last Login Time      720
Salary               995
Bonus %              971
Senior Management      2
Team                  10
dtype: int64
```

Handling missing values

In [57]:
```
data2.isnull().sum()
```

Out[57]:
```
First Name           67
Gender              145
Start Date            0
Last Login Time       0
Salary                0
Bonus %               0
Senior Management    67
Team                 43
dtype: int64
```

In [59]:
```
#Now, let's try to fill in the missing values of gender with the string "No Gender".
data2["Gender"].fillna("No Gender", inplace = True)

data2.isnull().sum()
```

Out[59]:
```
First Name           67
Gender                0
Start Date            0
Last Login Time       0
Salary                0
Bonus %               0
Senior Management    67
Team                 43
dtype: int64
```

In [60]:
```
# Now, Let's fill the senior management with the mode value.
mode = data2['Senior Management'].mode().values[0]
data2['Senior Management']= data2['Senior Management'].replace(np.nan, mode)

data2.isnull().sum()
```

Out[60]:
```
First Name           67
Gender                0
Start Date            0
Last Login Time       0
Salary                0
Bonus %               0
Senior Management     0
Team                 43
dtype: int64
```

In [61]:
```
#Now for the first name and team, we cannot fill the missing values with arbitrary data, so,
#let's drop all the rows containing these missing values.
# axis=0 for rows
# axis=1 for columns
data2 = data2.dropna(axis = 0, how ='any')#is used to remove any rows containing missing or NaN

print(data2.isnull().sum())
```

```
First Name           0
Gender               0
Start Date           0
Last Login Time      0
Salary               0
Bonus %              0
Senior Management    0
Team                 0
dtype: int64
```

Data encoding

There are some models like Linear Regression which does not work with categorical dataset in that case we should try to encode categorical dataset into the numerical column. we can use different methods for encoding like Label encoding or One-hot encoding.

In [62]:
```
from sklearn.preprocessing import LabelEncoder
# create an instance of LabelEncoder
le = LabelEncoder()

# fit and transform the "Senior Management"
# column with LabelEncoder
data2['Gender'] = le.fit_transform\
                  (data2['Gender'])
data2.head()
```

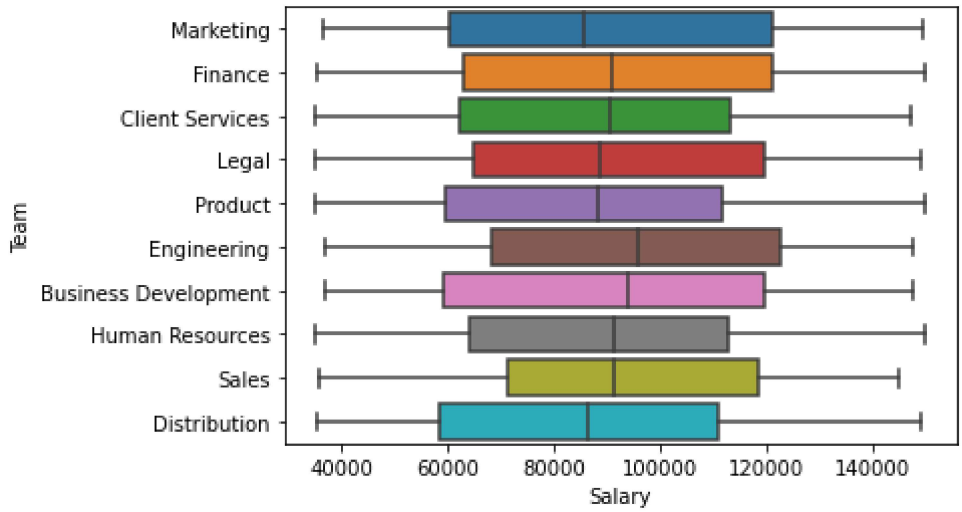
Out[62]:

	First Name	Gender	Start Date	Last Login Time	Salary	Bonus %	Senior Management	Team
0	Douglas	1	1993-08-06	12:42 PM	97308	6.945	True	Marketing
2	Maria	0	1993-04-23	11:17 AM	130590	11.858	False	Finance
3	Jerry	1	2005-03-04	1:00 PM	138705	9.340	True	Finance
4	Larry	1	1998-01-24	4:47 PM	101004	1.389	True	Client Services
5	Dennis	1	1987-04-18	1:35 AM	115163	10.125	False	Legal

Data Visualization

In [64]:
```
#histogram can be used for both uni and bivariate analysis.
# sns.histplot(x='Salary', data=data2, )
# plt.show()
```

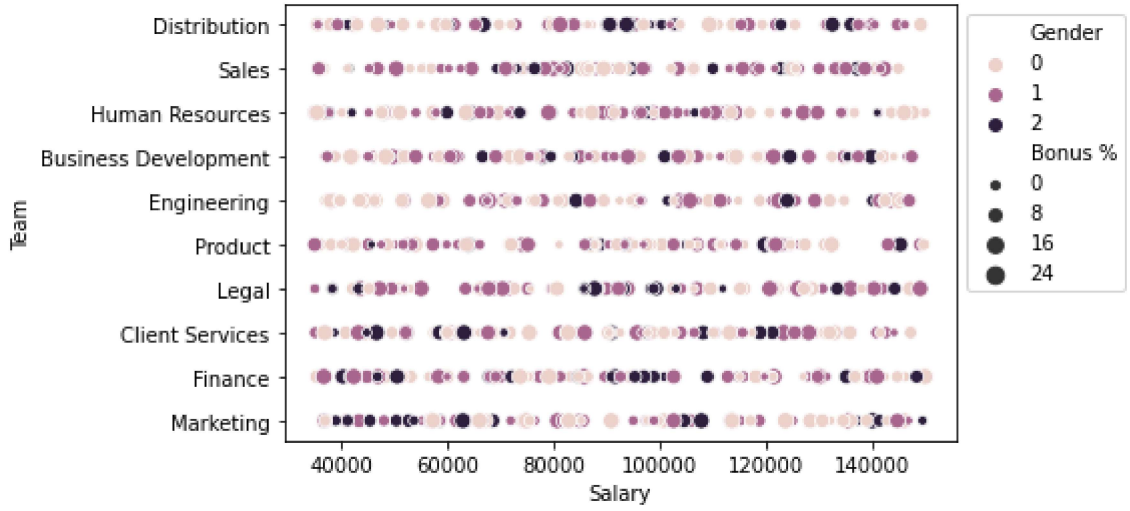
In [65]:
```
#Boxplot
#It can also be used for univariate and bivariate analyses.
sns.boxplot( x="Salary", y='Team', data=data2, )
plt.show()
```



In [66]:
```
# scatter bx plot
sns.scatterplot( x="Salary", y='Team', data=data2,
                hue='Gender', size='Bonus %')

# Placing Legend outside the Figure
plt.legend(bbox_to_anchor=(1, 1), loc=2)

plt.show()
```



more analysis

In [69]:
```
data2.shape
```

Out[69]: (899, 8)

In [70]:
```
data2.duplicated().sum()# duplicate rows
```

Out[70]:
```
0
```

SO WITH THIS WE BID END TO OUR EDA,NOW WE HAVE ENOUGH KNOWLEDGE TO PERFORM EDA ON ANY DATASET...

Exploratory Data Analysis Examples

Clinical Trial

The open-access, peer-reviewed scientific journal PLoS ONE published a clinical group study in which researchers used exploratory data analysis to identify outliers in the patient population and verify their homogeneity.

The scientists classified the patients participating in the study into forty attributes, including age and gender. EDA helped them determine that female groups in the study were more homogeneous than their male counterparts. This prompted the researchers to conduct separate medical tests for the male groups to avoid false findings in the clinical trial.

Retail

For example, an online store sells various types of footwear, such as sandals, sneakers, dress shoes, hiking boots, and formal shoes.

Exploratory data analysis can enable analysts to represent different sales trends graphically and visualize data related to best-selling product categories, buyer demographics and preferences, customer spending patterns, and units sold over a certain period.

Without EDA, this would not have been possible.

In [ ]:
```

```